



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Sharan Subramanian
July 11, 2025



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion

Executive Summary

- Summary of Methodologies:
 - Data Collection via API
 - Data Collection via Web Scraping
 - Data Wrangling
 - Exploratory Data Analysis (EDA) using SQL
 - EDA and Visualization with Pandas & Matplotlib
 - Interactive Visual Analytics with Folium
 - Interactive Dashboard with Plotly Dash
 - Machine Learning Classification for Predictive Analysis
- Summary of Results:
 - EDA Results
 - Interactive Analytics through Screenshots
 - Predictive Analysis Results

Introduction

- Project Background and Context:
 - SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars while other providers cost upward of 165 million dollars each. The savings is mainly because SpaceX can reuse the first stage. Therefore if we can determine if the first stage will land, we can determine the cost of a launch, which can be useful if an alternate company wants to bid against SpaceX for a rocket launch.
- Questions to Address:
 - What factors influence if a rocket will land successfully or not?
 - How do different launch sites affect the landing outcome?
 - Which classification model performs the best at predicting the landing outcome?

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - SpaceX data was collected from the SpaceX API and collected through web scraping from Wikipedia.
- Perform data wrangling
 - Data was cleaned, formatted, and transformed to apply training labels and prepare for analysis.
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - How to build, tune, evaluate classification models.

Data Collection

- Data Collection Steps (Using API):
 - Sent a GET request to the SpaceX API.
 - Converted the corresponding content to JSON form to then convert it to a Pandas data frame.
 - Used `.json()` to decode the response to a JSON
 - Used `.json_normalize()` to convert to a pandas data frame.
 - Conducted some simple data wrangling and cleaning to prepare the data for future analysis.
 - Filtered data to only include Falcon 9 launches
 - Replaced missing values
- Data Collection Steps (Using Web Scraping):
 - Sent an HTTP GET method to request the Falcon 9 Launch HTML page.
 - Created a BeautifulSoup object to store this response
 - Extracted column and variable names from the HTML table header.
 - Created a data frame by parsing the launch HTML tables.

Data Collection – SpaceX API

- The images to the right show the Python commands for the GET request sent, conversion from JSON to Pandas data frame, filtering of data, and wrangling of data.
- Link: [SpaceX API Notebook](#)

Now let's start requesting rocket launch data from SpaceX API with the following URL:

```
In [6]: spacex_url="https://api.spacexdata.com/v4/launches/past"
```

```
In [7]: response = requests.get(spacex_url)
```

Now we decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

```
In [20]: # Use json_normalize meethod to convert the json result into a dataframe
data = pd.json_normalize(response.json())
```

Finally we will remove the Falcon 1 launches keeping only the Falcon 9 launches. Filter the data dataframe using the `BoosterVersion` column to only keep the Falcon 9 launches. Save the filtered data to a new dataframe called `data_falcon9`.

```
In [44]: # Hint data['BoosterVersion']!='Falcon 1'
data_falcon9 = data[data['BoosterVersion'] == 'Falcon 9']
data_falcon9.head()
```

Calculate below the mean for the `PayloadMass` using the `.mean()`. Then use the mean and the `.replace()` function to replace `np.nan` values in the data with the mean you calculated.

```
In [55]: # Calculate the mean value of PayloadMass column
payloadmass_mean = data_falcon9['PayloadMass'].mean()
# Replace the np.nan values with its mean value
data_falcon9['PayloadMass'] = data_falcon9['PayloadMass'].replace(np.nan, payloadmass_mean)
data_falcon9.isnull().sum()
```


Data Collection – Web Scraping

- The images to the right show the Python commands for the GET request sent and BeautifulSoup object created, column names extracted, and new data frame created.
- Link: [SpaceX Web Scraping Notebook](#)

First, let's perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response.

```
In [18]: # use requests.get() method with the provided static_url
# assign the response to a object
response = requests.get(static_url)
```

Create a BeautifulSoup object from the HTML response

```
In [19]: # Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(response.text)
```

Next, we just need to iterate through the <th> elements and apply the provided extract_column_from_header() to extract column name one by one

```
In [24]: column_names = []

# Apply find_all() function with `th` element on first_launch_table
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the Non-empty column name ('if name is not None and len(name) > 0') into a list called column_names

th_elements = first_launch_table.find_all('th')
for th in th_elements:
    name = extract_column_from_header(th)
    if name is not None and len(name) > 0:
        column_names.append(name)
```

After you have fill in the parsed launch record values into launch_dict , you can create a dataframe from it.

```
In [40]: df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
```

Data Wrangling

- Data Wrangling:
 - Basic exploratory data analysis was conducted to find different patterns and determine the training labels for training supervised models.
 - These included:
 - Frequency of launches per launch site
 - Frequency of each orbit type
 - Frequency of each launch outcome
 - Creating a landing outcome label, called 'Class', was created from the 'Outcome' column in the data set.
 - 1 meant the booster landed successfully
 - 0 meant the booster did not land successfully
- Link: [SpaceX Data Wrangling Notebook](#)

EDA with Data Visualization

- Data Visualization:
 - To do some preliminary analysis, three types of graphs were created to find potential relationships between variables:
 - Scatter Point Chart
 - Multiple scatter point charts were created, specifically:
 - Flight Number vs Launch Site
 - Payload Mass vs Launch Site
 - Flight Number vs Orbit
 - Payload Mass vs Orbit
 - Bar Plot
 - A bar plot was created to compare the orbit type to success rate.
 - Line Plot
 - A line plot was created to find any trends between year and success rate.
- Link: [SpaceX EDA with Data Visualizations Notebook](#)

EDA with SQL

- SQL:
 - The SpaceX data set was loaded into a Db2 database to perform SQL queries.
 - These SQL queries included:
 - Finding the names of unique launch sites
 - Finding the average payload mass carried by a specific booster
 - Finding the total number of successful and unsuccessful mission outcomes
- Link: [SpaceX EDA with SQL Notebook](#)

Build an Interactive Map with Folium

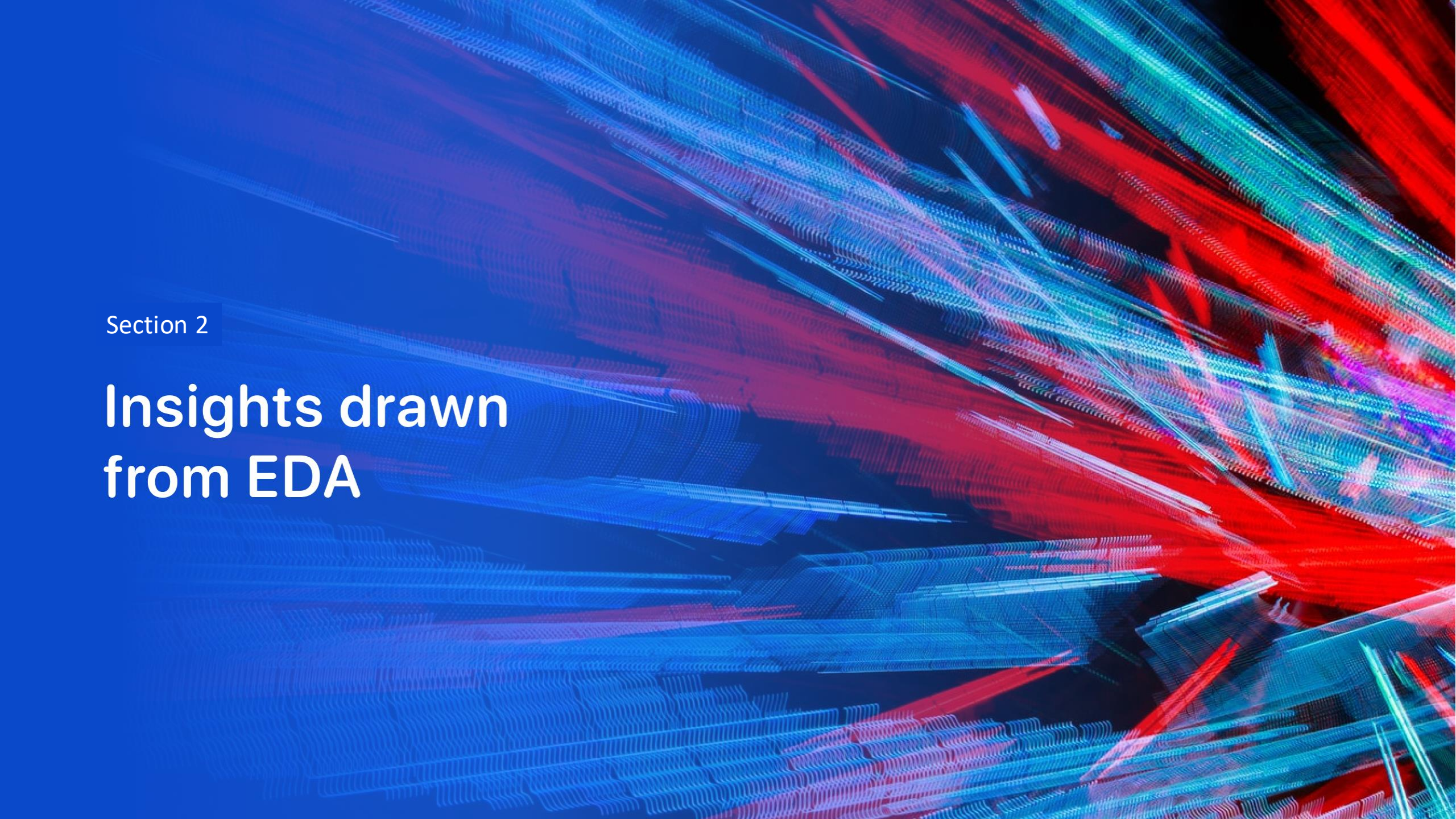
- Interactive Map:
 - Several map objects were created and added to a folium map to find potential geographical insights about launch sites.
 - These included:
 - Folium markers/circles for the launch sites
 - Folium markers/circles to highlight if a launch was a success or a failure
 - Green meant a success
 - Red meant a failure
 - Folium lines to calculate distances between launch sites and nearby proximities
- Link: [SpaceX Interactive Map with Folium Notebook](#)

Build a Dashboard with Plotly Dash

- Interactive Dashboard:
 - A dropdown menu was created to allow users to select either all launch sites or a specific one.
 - A payload range slider was added to enable users to choose their desired payload mass range.
 - A pie chart shows the total number of successful launches across all launch sites or the user-selected site.
 - A scatter chart displays the correlation between payload mass and launch success.
- Link: [SpaceX Interactive Dashboard with Plotly Notebook](#)

Predictive Analysis (Classification)

- Predictive Analysis:
 - The SpaceX data set was loaded using Pandas and NumPy.
 - The data was then split into a training set and a testing set.
 - Using GridSearchCV, we created four different model objects to find the best hyperparameters for each algorithm.
 - Logistic Regression
 - Support Vector Machine
 - Decision Tree
 - K Nearest Neighbors
 - The accuracies of each model were calculated and compared to determine the best-performing classification model.
- Link: [SpaceX Predictive Analysis Notebook](#)

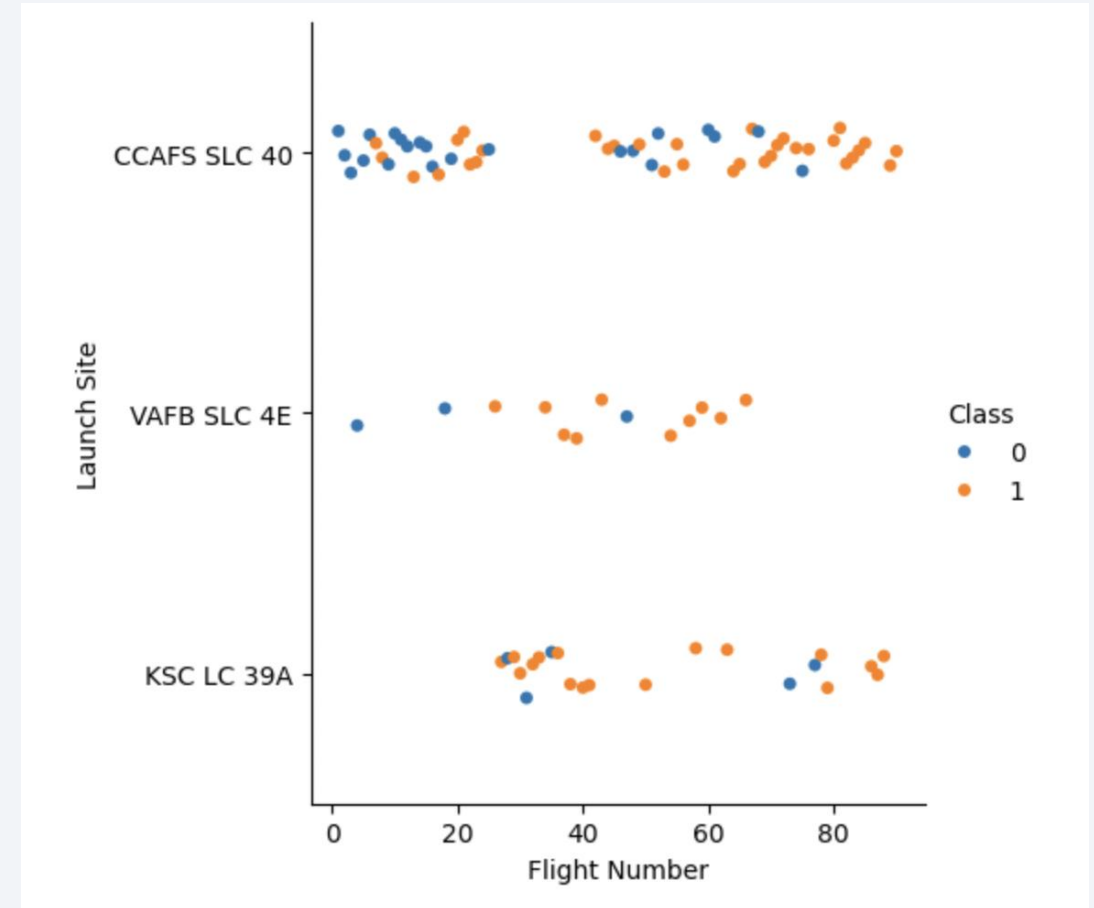
The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is high-tech and digital.

Section 2

Insights drawn from EDA

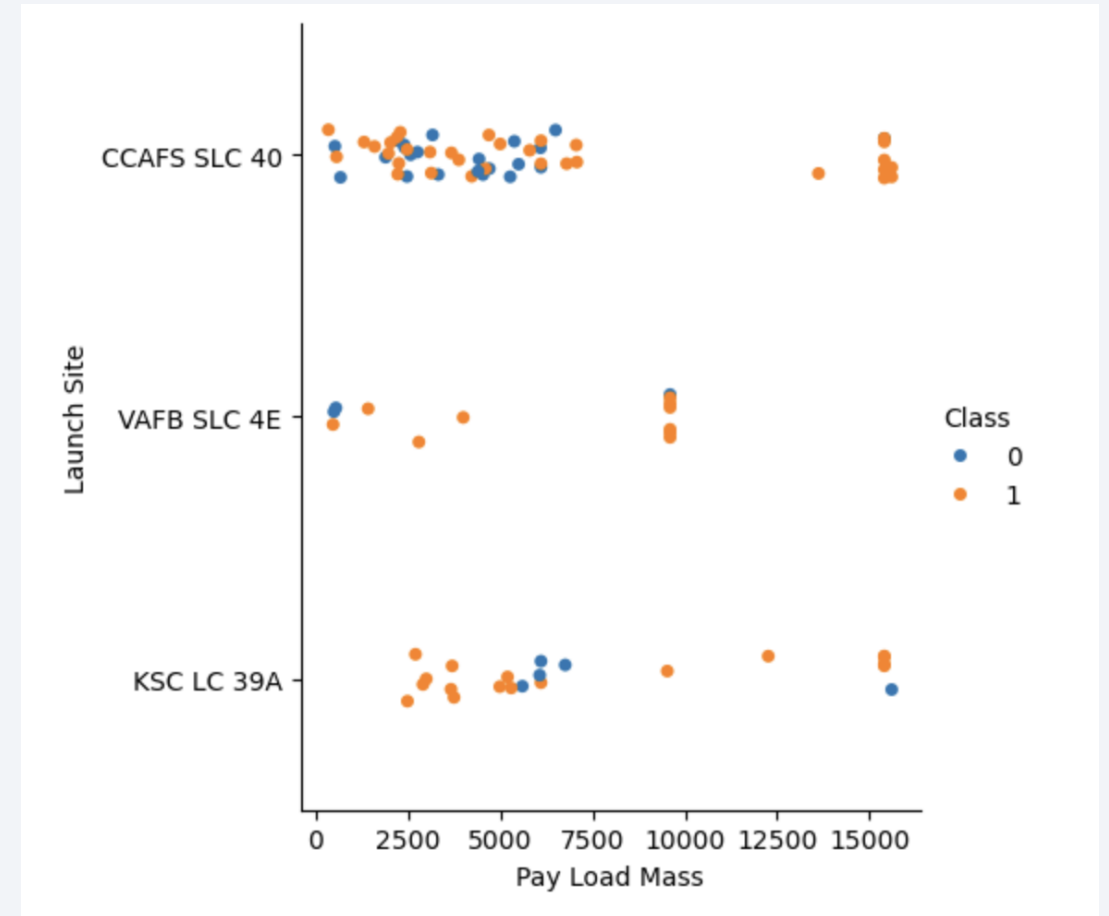
Flight Number vs. Launch Site

- The image to the right shows the scatter point chart between Flight Number and Launch Site.
- From the graph, we can see that the success rate increases as the flight number increases for all sites.
- For the SLC 40 launch site, there is a 100% success rate when the flight number is more than 75 (give or take).
- There are no launches for the SLC 4E launch site when the flight number is more than 65 (give or take).



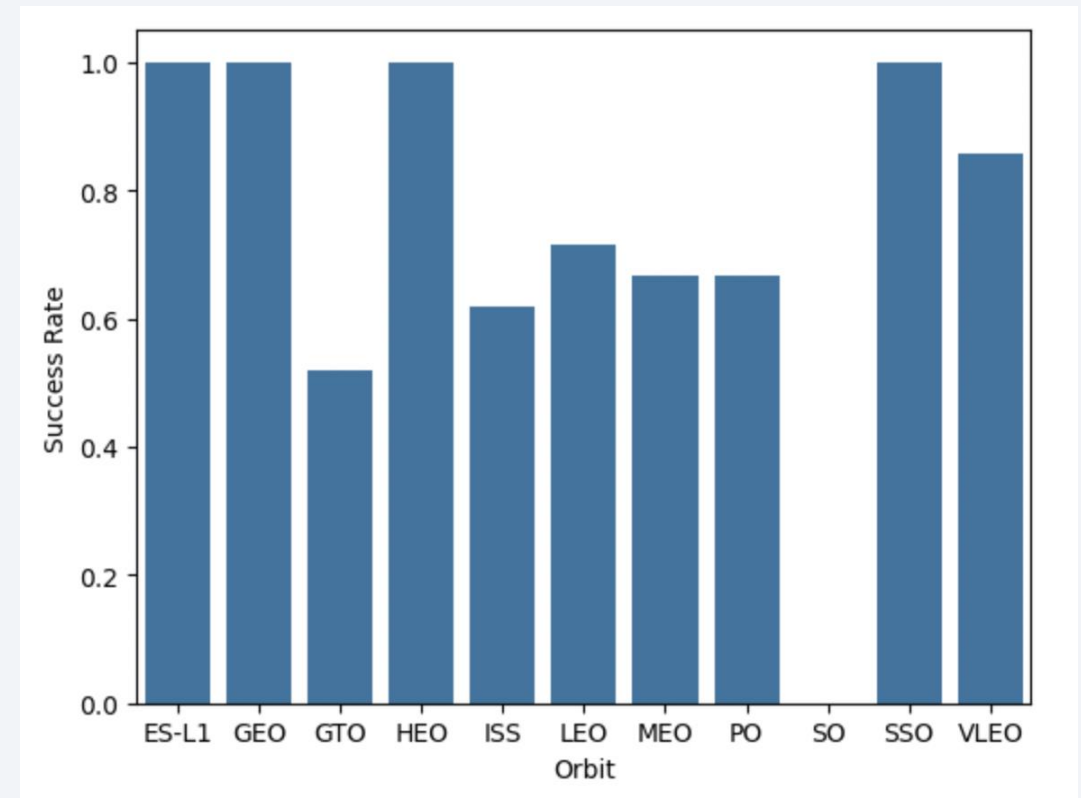
Payload vs. Launch Site

- The image to the right shows the scatter point chart between Payload Mass and Launch Site.
- The graphs reveals how success rate increases as the payload mass increases for all launch sites.
- For the SLC 40 launch site, there is a 100% success rate when the payload mass is more than 7,500.
- The SLC 4E launch site has no launches at all when the payload mass is more than 10,000.



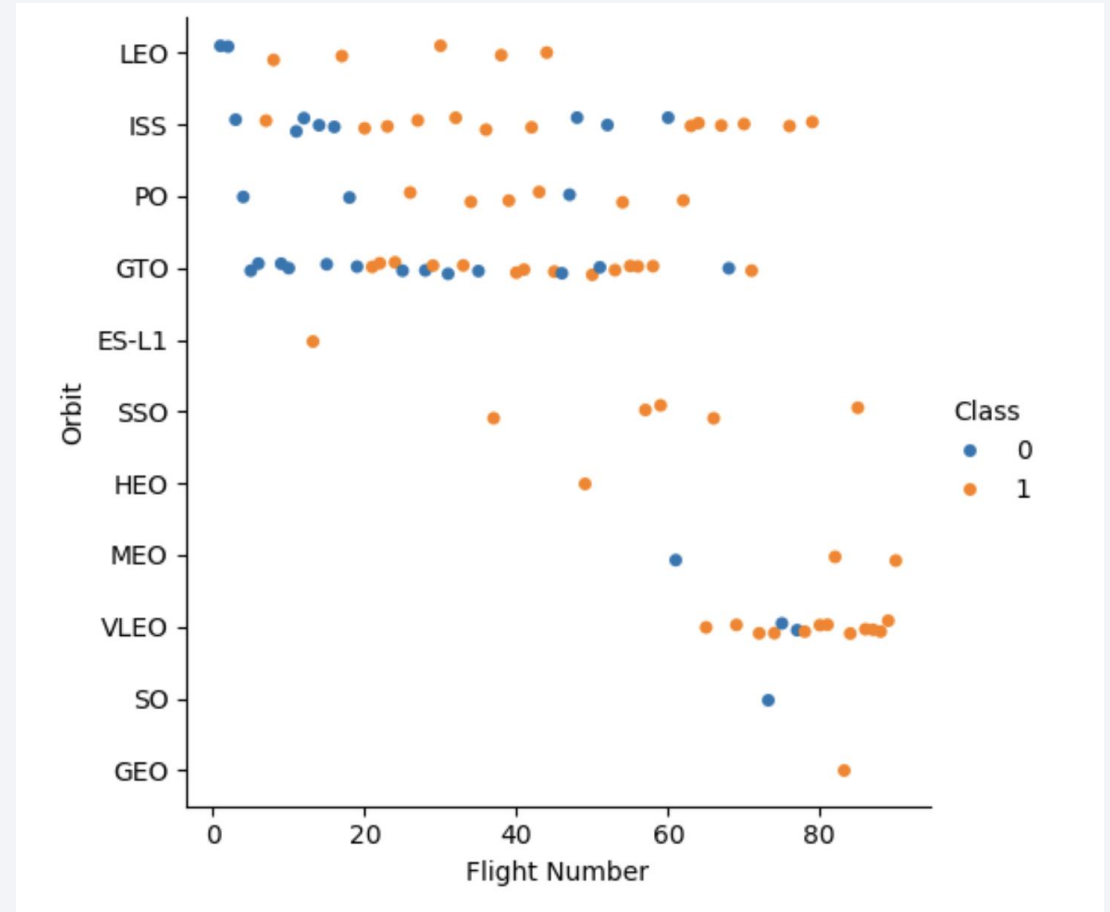
Success Rate vs. Orbit Type

- The image to the right shows a bar chart showing the relationship between Orbit Type and Success Rate.
- The chart shows how the success rate varies depending on the kind of orbit.
- Specifically, for instance, the orbit type SSO has a 100% success rate while the GTO orbit has only half of that rate.
- Out of all the orbit types, ES-L1, GEO, HEO, and SSO are the most successful.



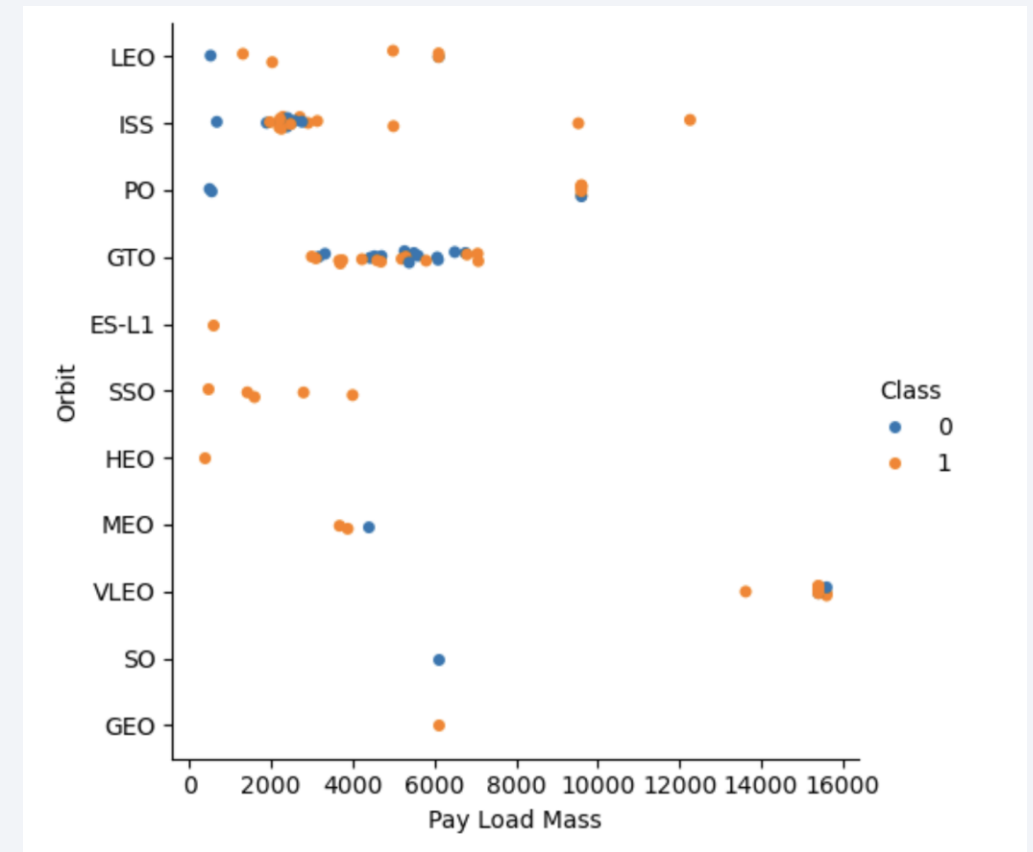
Flight Number vs. Orbit Type

- The image to the right shows a scatter point chart between Flight Number and Orbit Type.
- When in an LEO orbit, there seems to be a relationship between flight number and the number of successful launches.
- Conversely, for GTO orbits, there does not appear to be any correlation between flight number and success rate.



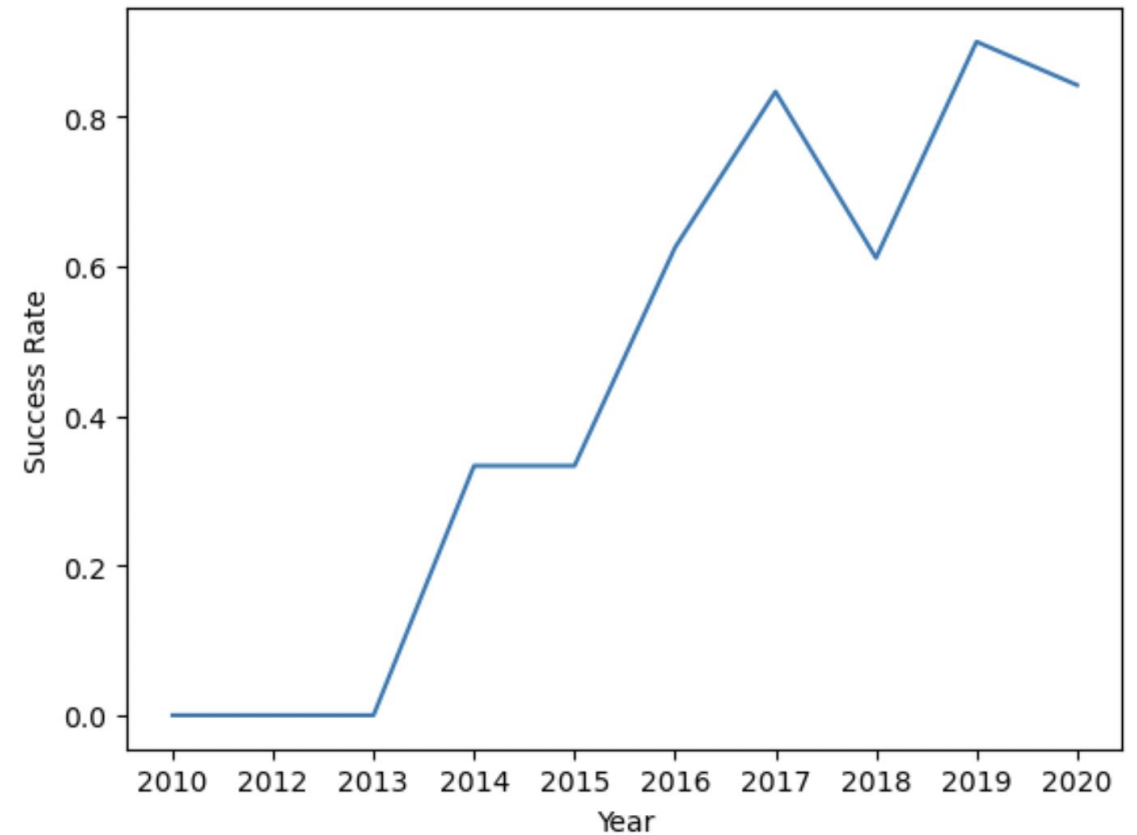
Payload vs. Orbit Type

- The image to the right shows a scatter point chart between Payload Mass and Orbit Type.
- The general trend is that heavy payload mass results in a higher chance of a successful landing, evident from orbit types ISS, PO, and VLEO.
- For GTO orbits, it is hard to distinguish any relationships since the success and failure landings are clustered together.



Launch Success Yearly Trend

- The image to the right shows a line plot between Year and Success Rate.
- Since 2013, the success rate has continued to increase, with a plateau from 2014 to 2015.
- However, in 2017, there was a considerable decrease in the success rate until it returned to increasing after 2018.



All Launch Site Names

- The image shows the SQL command and output that displays the names of the unique launch sites.

```
In [13]: %sql SELECT DISTINCT Launch_Site FROM SPACEXTBL
* sqlite:///my_data1.db
Done.
Out[13]: Launch_Site
         CCAFS LC-40
         VAFB SLC-4E
         KSC LC-39A
         CCAFS SLC-40
```

Launch Site Names Begin with 'CCA'

- The image shows the SQL command and output where launch sites begin with 'CCA'.

```
In [15]: %sql SELECT * FROM SPACEXTBL WHERE Launch_Site LIKE 'CCA%' LIMIT 5
* sqlite:///my_data1.db
Done.
```

```
Out[15]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (p
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (p
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	N
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	N
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	N

Total Payload Mass

- The image shows the SQL command and output that calculates the total payload carried by boosters from NASA.

```
In [26]: %sql SELECT SUM(PAYLOAD_MASS_KG_) AS total_payload_mass FROM SPACEXTBL WHERE CUSTOMER = 'NASA (CRS)'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[26]: total_payload_mass  
         45596
```

Average Payload Mass by F9 v1.1

- The image shows the SQL command and output that calculates the average payload mass carried by booster version F9 v1.1.

```
In [28]: %sql SELECT AVG(PAYLOAD_MASS_KG_) AS average_payload_mass FROM SPACEXTBL WHERE Booster_Version LIKE 'F9 v1.1%'
* sqlite:///my_data1.db
Done.
Out[28]: average_payload_mass
          2534.6666666666665
```

First Successful Ground Landing Date

- The image shows the SQL command and output that finds the dates of the first successful landing outcome on ground pad.

```
In [29]: %sql SELECT MIN(DATE) FROM SPACEXTBL WHERE Landing_Outcome LIKE '%ground pad%'
* sqlite:///my_data1.db
Done.
Out[29]: MIN(DATE)
          2015-12-22
```

Successful Drone Ship Landing with Payload between 4,000 and 6,000

- The image shows the SQL command and output of the names of boosters which have successfully landed on drone ship and had payload mass greater than 4,000 but less than 6,000.

```
In [31]: %sql SELECT Booster_Version FROM SPACEXTBL WHERE Landing_Outcome = 'Success (drone ship)' and PAYLOAD_MASS_KG_ >
* sqlite:///my_data1.db
Done.
Out[31]: 

| Booster_Version |
|-----------------|
| F9 FT B1022     |
| F9 FT B1026     |
| F9 FT B1021.2   |
| F9 FT B1031.2   |


```

Total Number of Successful and Failure Mission Outcomes

- The image shows the SQL command and output to calculate the total number of successful and failure mission outcomes.

```
In [35]: %sql SELECT Mission_Outcome, COUNT(*) AS TOTAL FROM SPACEXTBL WHERE Mission_Outcome LIKE '%Success%' OR Mission_C
* sqlite:///my_data1.db
Done.
```

Out[35]:

Mission_Outcome	TOTAL
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

- The image shows the SQL command and output that shows the names of the booster which have carried the maximum payload mass.

```
In [37]: %sql SELECT Booster_Version, PAYLOAD_MASS_KG_ AS Max_Payload_Mass FROM SPACEXTBL WHERE PAYLOAD_MASS_KG_ = (SELE
* sqlite:///my_data1.db
Done.
Out[37]:
```

Booster_Version	Max_Payload_Mass
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

2015 Launch Records

- The image shows the SQL command and output that lists the failed landing outcomes in drone ship, their booster versions, and launch site names for in year 2015.

```
In [44]: %sql SELECT substr(Date, 6, 2) AS Month, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTBL WHERE Landi
* sqlite:///my_data1.db
Done.
Out[44]:
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40
06	Precluded (drone ship)	F9 v1.1 B1018	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- The image shows the SQL command and output that ranks the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
In [43]: %sql SELECT Landing_Outcome AS count_outcome FROM SPACEXTBL WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY Landing_Outcome ORDER BY count_outcome DESC
```

* sqlite:///my_data1.db
Done.

```
Out[43]:
```

count_outcome
Uncontrolled (ocean)
Success (ground pad)
Success (drone ship)
Precluded (drone ship)
No attempt
Failure (parachute)
Failure (drone ship)
Controlled (ocean)

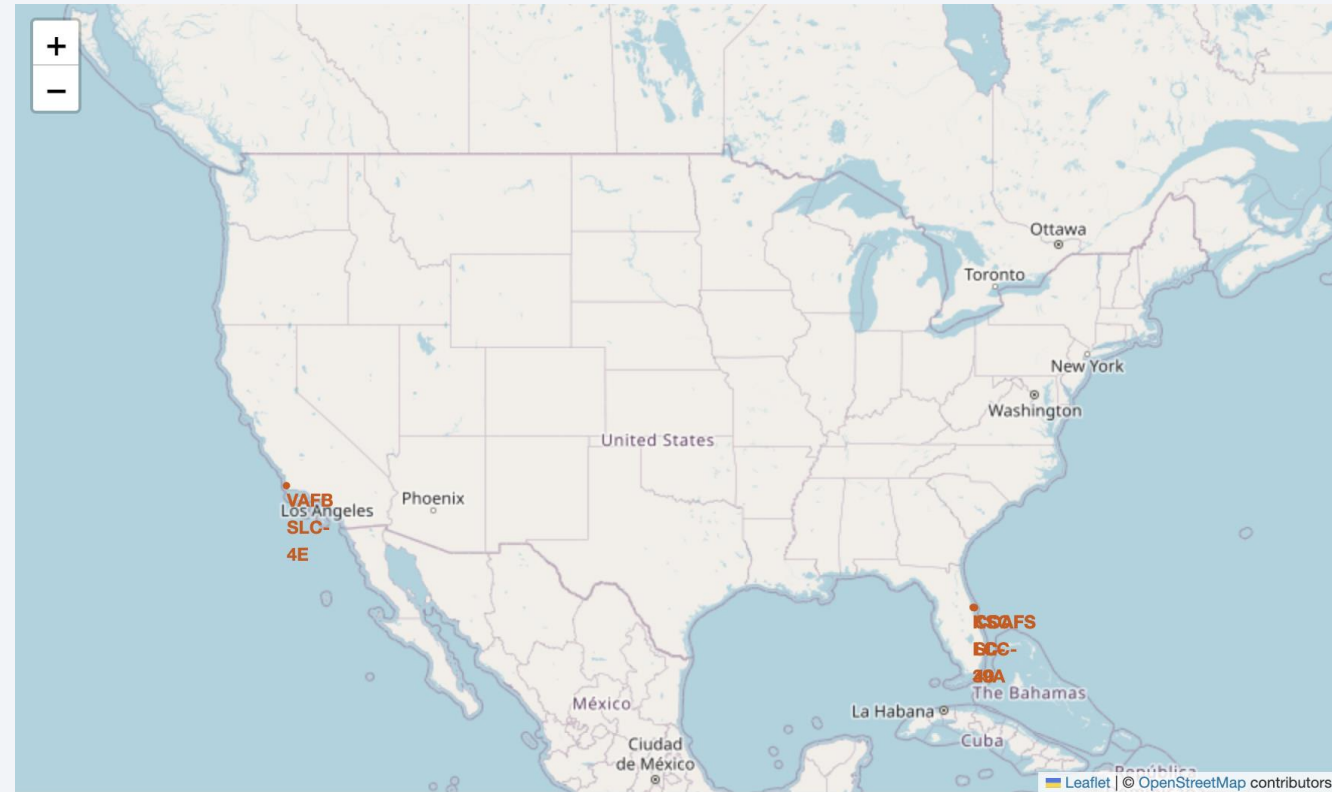
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

Launch Sites Proximities Analysis

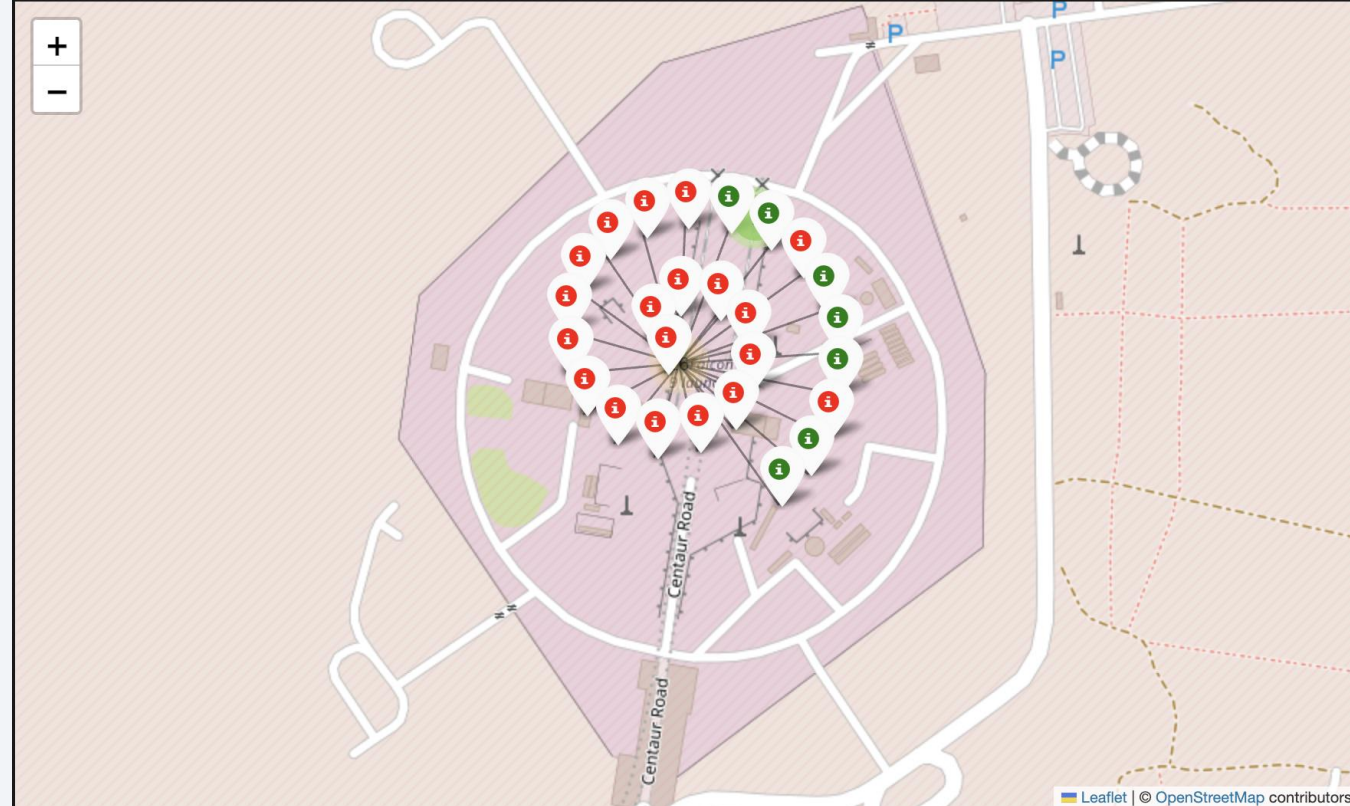
Launch Sites Locations on Map

- The image shows a map of the United States that includes markers of the SpaceX launch sites.
- Since the map is interactive, you can zoom in on the specific sites or any relevant nearby areas.



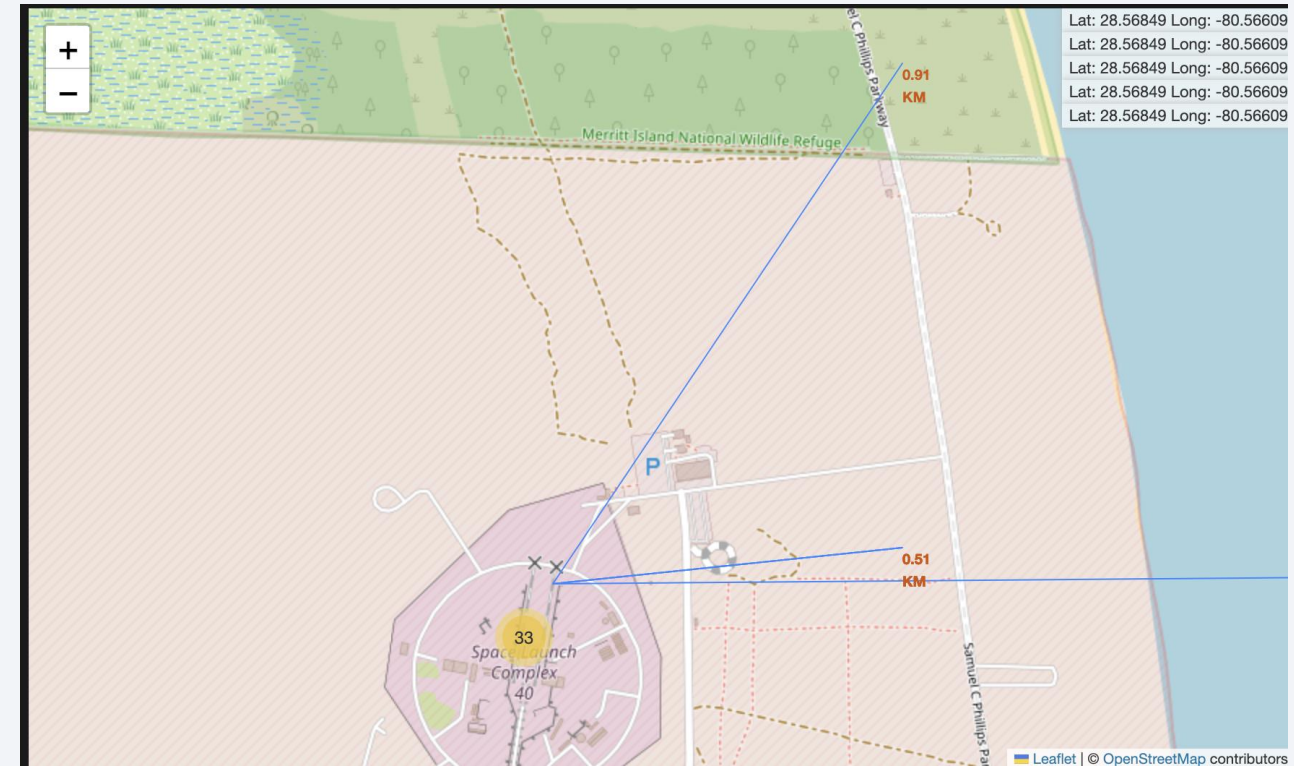
Successful & Failed Launches

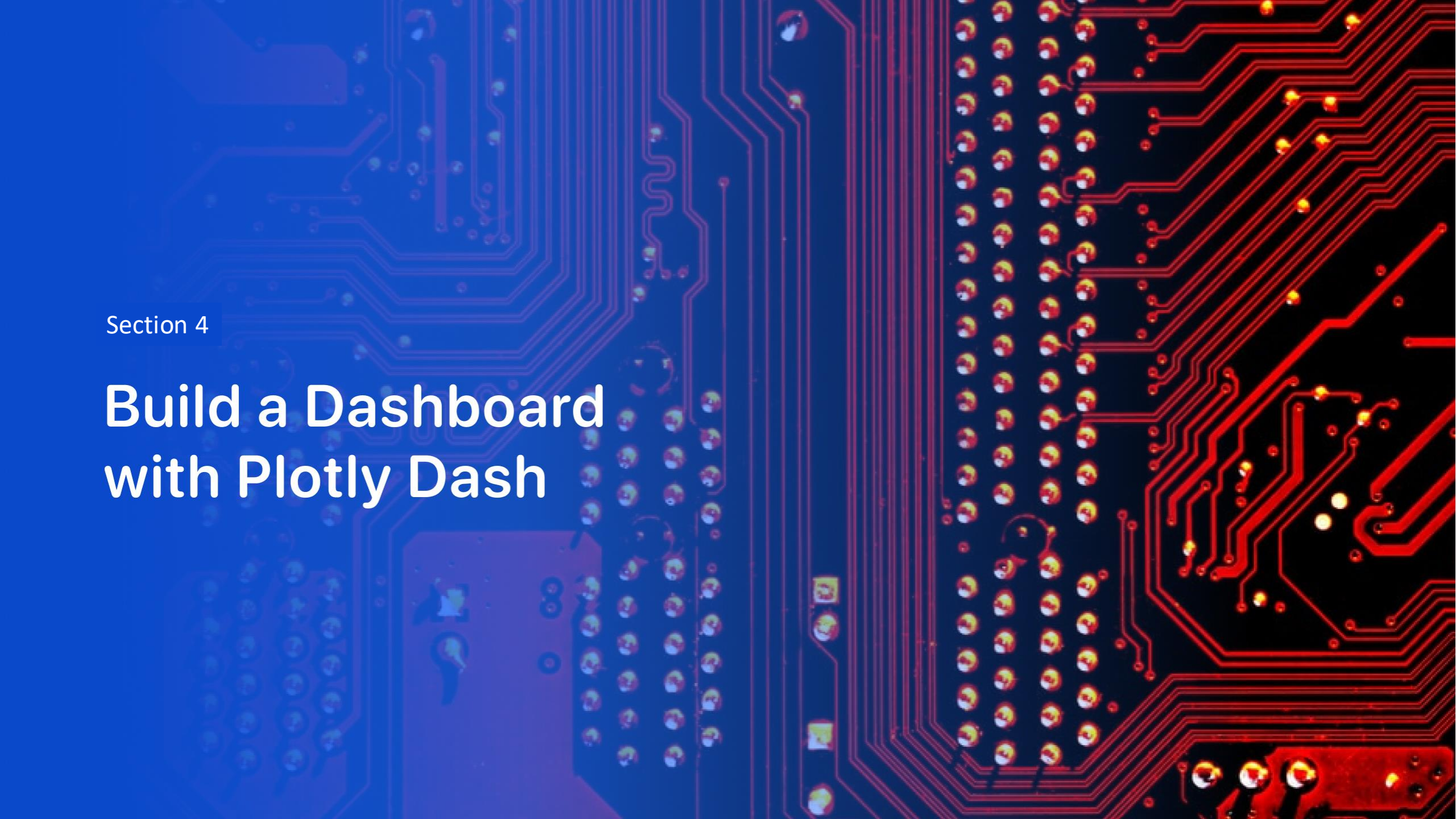
- The image shows a specific launch site of red and green markers.
 - A red marker indicates a failed launch
 - A green marker indicates a successful launch
- This is just one launch site – the same can be done for other sites to see which sites are more successful than others.



Distances from Launch Sites to Nearby Locations

- The image displays a line drawn from the launch site to a nearby location – in this case a railroad.
- The distance (in kilometers), is shown to indicate the proximity between the launch site and location.
- This process can be repeated similarly for other launch sites by inputting the location coordinates. The map will then automatically generate a line and display the corresponding distance.





Section 4

Build a Dashboard with Plotly Dash

Pie Chart: Successful Launches by Site

- Below is a pie chart showing the distribution of the successful landings for each launch site.
- We can see that the KSC LC-39A launch site has the highest success rate, at 41.7%.

Total Number of Successes Per Launch Site



Success vs Failure for KSC LC-39A

- Below is a pie chart of the launch site with the highest landing success ratio.
- The KSC LC-39A site has a success rate of around 76.9%, with a failure rate of only 23.1%.

Total Number of Successes and Failures for site KSC LC-39A



Payload vs Launch Outcome Scatter Plot

- A scatter plot of Payload vs Launch Outcome for all sites is shown below, where the payload range is between 0 kg and 5,000 kg.
- From this plot, it's evident that there is a somewhat equal number of successful and failed launches.



Payload vs Launch Outcome Scatter Plot

- Now, another scatter plot of Payload vs Launch Outcome for all sites is shown below, except this time the payload range is between 5,000 kg and 10,000 kg.
- Clearly, there are fewer points on this plot compared to the previous one (especially more failed launches), highlighting that launches with lower payloads tend to have a higher success rate than those with heavier payloads.



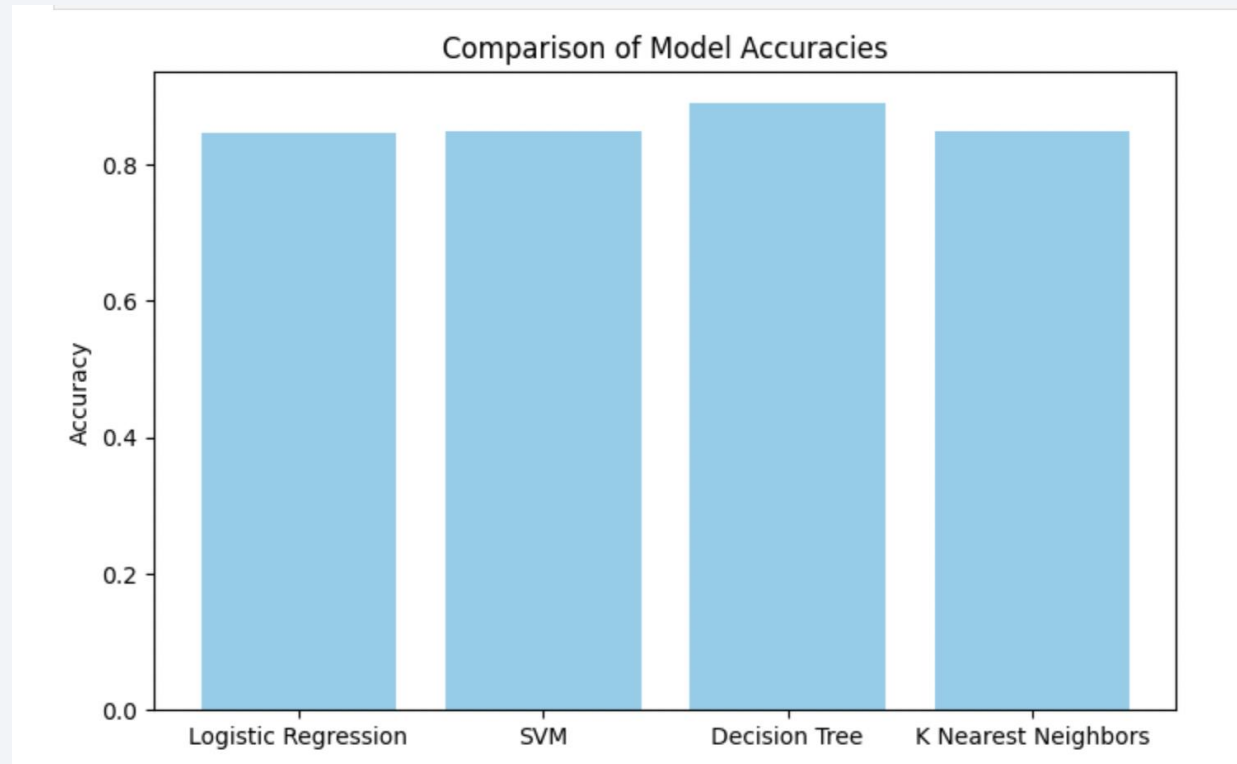


Section 5

Predictive Analysis (Classification)

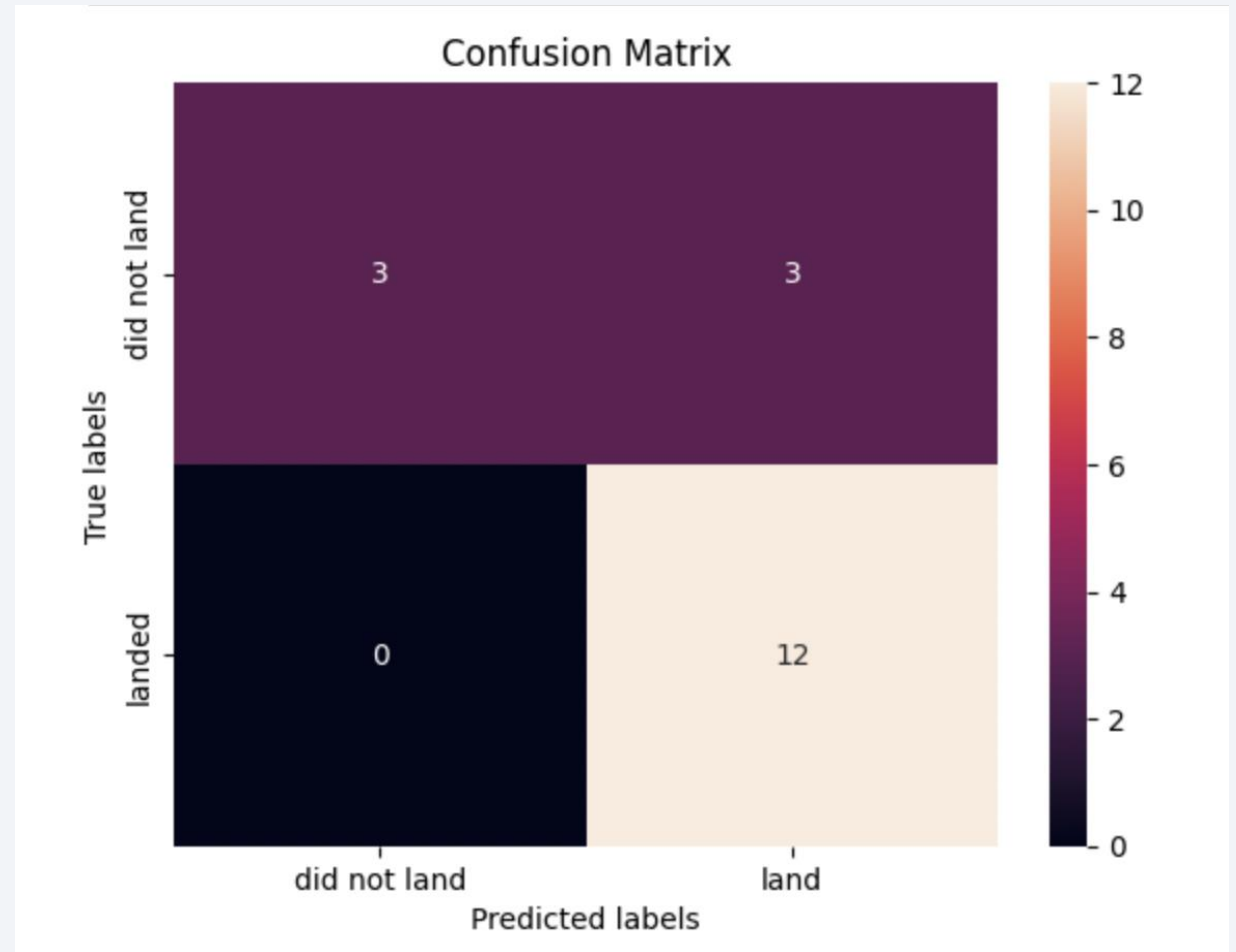
Classification Accuracy

- The bar plot below displays the different models used and their corresponding accuracy.
- The decision tree model has the highest accuracy of the four, with the other three relatively close to the same accuracy.



Confusion Matrix

- The image is a confusion matrix for the decision tree model, the one with the highest accuracy.
- The model correctly predicted 12 landing outcomes as landed (true positive).
- However, the model incorrectly predicted 3 landing outcomes as landed when they actually did not land (false positive).



Conclusions

- The following can be concluded:
 - As the flight number increases, the likelihood of a successful landing also increases across all launch sites.
 - The orbit types with the highest success rate are ES-L1, GEO, HEO, and SSO.
 - KSC LC-39 is the most successful launch site among the four.
 - The decision tree model resulted in the highest accuracy in classifying launches as a success or failure.

Thank you!

